

Tổng Hợp Kiến Thức

1. HTTP Stateless

- **Định nghĩa:** HTTP là giao thức **không trạng thái** (stateless). Mỗi request là độc lập, server không tự nhớ các request trước từ cùng client.
- **Vấn đề:** Cần cơ chế để lưu trạng thái (đăng nhập, giỏ hàng, tùy chọn người dùng).
- **Giải pháp:** Cookie, Session, hoặc JWT.

2. Cookie

- **Định nghĩa:** Dữ liệu nhỏ server gửi về client, lưu trong trình duyệt, tự động gửi lại trong các request sau đến cùng domain.
- **Mục đích:** Nhận diện người dùng, lưu tùy chọn nhỏ (ngôn ngữ, theme).
- **Middleware:** `cookie-parser` để parse header `Cookie` thành `req.cookies`.
- **Tạo Cookie:**

```
res.cookie('name', 'value', {
  maxAge: 900000, // Thời gian sống (ms)
  httpOnly: true, // Ngăn JS client truy cập (chống XSS)
  secure: process.env.NODE_ENV === 'production', // Chỉ gửi qua HTTPS
  sameSite: 'strict' // Chống CSRF
});
```

- **Đọc Cookie:**

```
const value = req.cookies.name;
```

- **Xóa Cookie:**

```
res.clearCookie('name', { httpOnly: true, secure: true, sameSite: 'strict' });
```

Tùy chọn quan trọng

- `maxAge` : Thời gian sống (ms).
- `expires` : Ngày hết hạn (Date).
- `httpOnly` : Ngăn JS client truy cập (chống XSS).
- `secure` : Chỉ gửi qua HTTPS.
- `sameSite` : `strict` (chỉ cùng site), `lax` (top-level navigation), `none` (cross-site, cần `secure`).
- `path` , `domain` : Xác định phạm vi cookie.

Lưu ý bảo mật

- **Luôn** dùng `httpOnly: true` cho cookie nhạy cảm (Session ID, token).
- **Luôn** dùng `secure: true` trong production (HTTPS).
- Dùng `sameSite: 'strict'` hoặc `'lax'` để chống CSRF.

3. Session

- **Định nghĩa:** Lưu trữ trạng thái người dùng **trên server**, dùng Session ID (lưu trong cookie) để nhận diện.
- **Mục đích:** Lưu thông tin nhạy cảm, dữ liệu lớn (đăng nhập, profile, giỏ hàng).
- **Middleware:** `express-session` + Session Store (ví dụ: `express-mysql-session`).

Cú pháp

- **Tạo bảng Session Store (MySQL):**

```
CREATE TABLE sessions (  
  session_id VARCHAR(128) COLLATE utf8mb4_bin NOT NULL PRIMARY KEY,  
  expires INT(11) UNSIGNED NOT NULL,  
  data MEDIUMTEXT COLLATE utf8mb4_bin  
);
```

- **Cấu hình `express-session` :**

```
const session = require('express-session');  
const MySQLStore = require('express-mysql-session')(session);  
  
const sessionStore = new MySQLStore({  
  host: 'localhost',  
  user: 'root',  
  password: '',  
  database: 'express_auth_demo',  
  clearExpired: true,  
  checkExpirationInterval: 1000 * 60 * 10, // 10 phút  
  expiration: 1000 * 60 * 60 * 24 // 1 ngày  
});  
  
app.use(session({  
  secret: 'your_secret_key',  
  store: sessionStore,  
  resave: false,  
  saveUninitialized: false,  
  cookie: {  
    httpOnly: true,  
    secure: process.env.NODE_ENV === 'production',  
    maxAge: 1000 * 60 * 60 * 24 // 1 ngày  
  }  
}));
```

- **Lưu dữ liệu Session:**

```
req.session.userId = user.id;  
req.session.data = { name: 'Alice' };
```

- **Đọc dữ liệu Session:**

```
const userId = req.session.userId;
```

- **Hủy Session:**

```
req.session.destroy(err => {  
  if (err) console.error(err);  
  res.clearCookie('connect.sid'); // Tên cookie mặc định  
});
```

Tùy chọn quan trọng

- `secret` : Chuỗi bí mật để ký Session ID.
- `resave` : `false` để tránh lưu lại session nếu không thay đổi.
- `saveUninitialized` : `false` để không tạo session cho request chưa xác thực.
- `cookie` : Tùy chọn cookie chứa Session ID (như `httpOnly` , `secure`).
- Store: `MemoryStore` (mặc định, không dùng production), `MySQLStore` , `Redis` ,...

Lưu ý

- **Production:** Dùng persistent store (MySQL, Redis) để scale ngang.
- Session ID cookie **phải** `httpOnly: true` và `secure: true` (HTTPS).
- Khớp `cookie.maxAge` với `store.expiration` .

4. JWT vs Cookie & Session

Đặc điểm	Cookie & Session (Stateful)	JWT (Stateless)
Lưu trạng thái	Server (MySQL, Redis)	Client (token)
Xác thực	Tìm session trong store	Xác minh chữ ký
Invalidation	Dễ (xóa session)	Khó (blacklist hoặc chờ hết hạn)
Scale	Cần chia sẻ store giữa server	Dễ, server độc lập
Server Load	Cao (truy vấn store)	Thấp (chỉ xác minh chữ ký)
Bảo mật	Dữ liệu an toàn trên server	Payload đọc được, không lưu dữ liệu bí mật
Lưu trữ client	Session ID (cookie)	Token (cookie, localStorage)

Khi nào dùng?

- **Session:** Web truyền thống, cần invalidation tức thời, dữ liệu lớn/bí mật.
- **JWT:** API, SPA, mobile, microservices, cần scale dễ, chấp nhận invalidation phức tạp.

Lưu trữ JWT

- **localStorage:** Dễ dùng, nhưng dễ bị XSS.
- **HTTP-only Cookie:** An toàn hơn (chống XSS), server quản lý, browser tự gửi.

Khuyến nghị: Lưu JWT trong **HTTP-only cookie** để tăng bảo mật.